🚀 Collections Framework Masterclass

Chapter 6 — Vector & Stack (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐☆

`Vector` and `Stack` are considered **legacy collection classes**, but they are still frequently asked in interviews because they help interviewers evaluate your understanding of the **evolution of the Java Collections Framework**, **thread safety**, and **legacy vs modern APIs**.

Although you may rarely use them in new applications, you **must know** how they work and why modern alternatives are preferred.

History of Vector

Before the Java Collections Framework (JDK 1.2), Java provided a few collection-like classes.

One of them was:

```
Vector
```

After JDK 1.2,

`Vector` was modified to implement the `List` interface.

That means today it behaves like a normal List while retaining its original synchronized behavior.

What is Vector?

`Vector` is a **synchronized**, **resizable array** implementation of the `List` interface.

Package

```
java.util.Vector
```

Declaration

```
public class Vector<E>
        extends AbstractList<E>
        implements List<E>,
                   RandomAccess,
                   Cloneable,
                   Serializable
```

Notice

Like ArrayList,

Vector implements

```
RandomAccess
```

because internally it also uses an array.

Position in Collection Hierarchy

```
Iterable
    ↑
Collection
    ↑
List
    ↑
Vector
    ↑
Stack
```

Characteristics of Vector

✅ Maintains insertion order

✅ Allows duplicates

✅ Allows null values

✅ Dynamic resizing

✅ Thread-safe

❌ Slower than ArrayList

❌ Legacy class

Internal Structure

Exactly like ArrayList,

Vector stores elements inside

```
Object[] elementData;
```

Example

```
+----+----+----+----+----+
| A  | B  | C  | D  | E  |
+----+----+----+----+----+
```

Internally

It is still an array.

Why is Vector Thread-Safe?

Interview Favorite ⭐⭐⭐⭐⭐

Every public method is synchronized.

Example

```
public synchronized boolean add(E e)
```

Meaning

Only one thread can modify the Vector at a time.

Example

```
Thread A

↓

Lock Vector

↓

Insert

↓

Unlock

↓

Thread B
```

This prevents concurrent modification issues.

Synchronization Cost

Interview Favorite ⭐⭐⭐⭐⭐

Why is Vector slower?

Because every operation requires

- Acquiring a lock
- Releasing the lock

Even if only one thread is using it.

Example

```
vector.add("Java");
```

Internally

```
Acquire Lock

↓
```

```
Insert

↓

Release Lock
```

This additional locking introduces overhead.

Creating a Vector

```
Vector<String> vector =
new Vector<>();
```

Constructors

Default Constructor

```
Vector<Integer> vector =
new Vector<>();
```

Default Capacity

```
10
```

Initial Capacity

```
Vector<Integer> vector =
new Vector<>(100);
```

Capacity

```
100
```

Capacity Increment

Unique feature of Vector.

```
Vector<Integer> vector =
new Vector<>(10,5);
```

Meaning

```
Initial Capacity

↓

10

Capacity Increment

↓

5
```

Growth

```
10

↓

15

↓

20

↓

25
```

Interview Favorite ⭐⭐⭐⭐⭐

Growth Algorithm

If no capacity increment is specified,

Vector doubles its capacity.

Example

```
10

↓

20

↓

40

↓

80
```

Unlike ArrayList

```
10

↓

15

↓

22

↓

33
```

ArrayList grows by approximately **1.5×**.

Vector doubles.

Size vs Capacity

Suppose

```
Vector<Integer> vector =
new Vector<>();
```

Add

```
5 elements
```

Result

```
Size

↓

5

Capacity

↓

10
```

Exactly like ArrayList.

Important Methods

add()

```
vector.add("Java");
```

Adds at end.

---

addElement()

Legacy method.

```
vector.addElement("Java");
```

Same purpose.

---

remove()

```
vector.remove(2);
```

Removes element.

---

removeElement()

Legacy version.

---

elementAt()

Instead of

```
vector.get(2);
```

Legacy API

```
vector.elementAt(2);
```

---

firstElement()

Returns first element.

```
vector.firstElement();
```

---

lastElement()

Returns last element.

```
vector.lastElement();
```

---

capacity()

Returns current capacity.

Example

```
System.out.println(
vector.capacity()
);
```

Interview Favorite ⭐⭐⭐⭐

ArrayList does **not** expose capacity publicly.

Vector Example

```
Vector<String> vector =
new Vector<>();

vector.add("Java");
vector.add("Spring");
vector.add("Docker");

System.out.println(vector);
```

Output

```
[Java, Spring, Docker]
```

Enumeration

Before Iterator,

Java used

```
Enumeration
```

Example

```
Enumeration<String> e =
vector.elements();

while(e.hasMoreElements())
{
    System.out.println(
        e.nextElement()
    );
}
```

Enumeration is still supported for backward compatibility.

Enumeration vs Iterator

Feature	Enumeration	Iterator
Legacy	Yes	No
Remove Elements	No	Yes
Forward Traversal	Yes	Yes
Fail-Fast	No	Yes

Interview Favorite ⭐⭐⭐⭐⭐

Stack

Now let's discuss Stack.

What is Stack?

`Stack` extends Vector.

Declaration

```
public
class Stack<E>
extends Vector<E>
```

Meaning

Every Vector method is available inside Stack.

LIFO Principle

Stack follows

```
LIFO

↓

Last In First Out
```

Example

```
Push

10

20

30

Stack

30

20

10
```

Pop

```
30
```

comes out first.

Stack Operations

push()

```
stack.push(10);
stack.push(20);
```

---

pop()

```
stack.pop();
```

Removes top element.

---

peek()

```
stack.peek();
```

Returns top element.

Does not remove it.

---

empty()

Checks whether stack is empty.

```
stack.empty();
```

---

search()

Returns distance from top.

Example

```
Stack<Integer> stack =
new Stack<>();

stack.push(10);
stack.push(20);
stack.push(30);

System.out.println(
stack.search(20)
);
```

Output

```
2
```

Stack Example

```
Stack<String> stack =
new Stack<>();

stack.push("Java");
stack.push("Spring");
stack.push("AWS");

System.out.println(stack.pop());

System.out.println(stack.peek());
```

Output

```
AWS

Spring
```

Why Stack is Legacy?

Interview Favorite ⭐⭐⭐⭐⭐

Because it extends Vector.

Therefore

Every operation is synchronized.

Extra overhead.

Modern Java recommends

```
ArrayDeque
```

instead.

ArrayDeque vs Stack

Feature	Stack	ArrayDeque
Thread-Safe	Yes (via Vector synchronization)	No
Legacy	Yes	No
Performance	Slower	Faster
Recommended	No	Yes

Interview Favorite ⭐⭐⭐⭐⭐

Why ArrayDeque is Preferred?

ArrayDeque

- Faster
- No synchronization overhead
- Better memory usage
- Designed specifically for Queue and Stack operations

Example

```
Deque<Integer> stack =
new ArrayDeque<>();

stack.push(10);
stack.push(20);

System.out.println(stack.pop());
```

Vector vs ArrayList

Feature	Vector	ArrayList
Thread-Safe	Yes	No
Internal Structure	Dynamic Array	Dynamic Array
Synchronization	Yes	No
Performance	Slower	Faster
Growth	2× (default)	~1.5×
Legacy	Yes	No
Capacity API	Yes	No

Time Complexity

Operation	Complexity
get()	O(1)
set()	O(1)
add()	O(1) Amortized
remove(index)	O(n)
contains()	O(n)
iteration	O(n)
push()	O(1)
pop()	O(1)
peek()	O(1)

Common Mistakes

Mistake 1

Using Stack in new projects.

Better

```
Deque<Integer> stack =
new ArrayDeque<>();
```

Mistake 2

Using Vector assuming it is always the correct thread-safe list.

Modern alternatives include

- `Collections.synchronizedList()`
- `CopyOnWriteArrayList`

depending on the use case.

Mistake 3

Thinking synchronization means "better."

Synchronization improves thread safety but also adds performance overhead.

Frequently Asked Interview Questions

Q1. Difference between Vector and ArrayList?

- Vector is synchronized.
- ArrayList is not synchronized.
- Vector is slower because of locking.
- Vector doubles capacity by default; ArrayList grows by approximately 1.5×.

Q2. Why is Vector slower?

Because every public method is synchronized, requiring lock acquisition and release.

Q3. What is the default capacity of Vector?

10.

Q4. What is Capacity Increment?

A Vector-specific feature that allows you to specify how much the internal array should grow when it becomes full.

Q5. What is Enumeration?

A legacy traversal interface introduced before `Iterator`. It supports forward traversal but cannot remove elements.

Q6. Why is Stack considered legacy?

Because it extends `Vector`, inherits synchronization overhead, and has been superseded by `Deque` implementations such as `ArrayDeque`.

Q7. Which should be used instead of Stack?

`ArrayDeque`.

Q8. Is Vector always the best choice for multi-threaded applications?

No.

Modern concurrent collections such as `CopyOnWriteArrayList` or properly synchronized wrappers are often better choices, depending on the workload.

Key Takeaways

- `Vector` is a synchronized, resizable array implementation of `List`.
- It uses an internal `Object[]` just like `ArrayList`.

- Every public method is synchronized, making it thread-safe but slower.
- Vector supports a unique `capacityIncrement` feature.
- `Stack` extends `Vector` and follows the LIFO principle.
- Modern Java recommends `ArrayDeque` instead of `Stack`.
- `Enumeration` is a legacy traversal mechanism; `Iterator` is the modern replacement.

Next Chapter

🚀 Chapter 7 – Set Interface (Complete Deep Dive)

We'll cover:

- What is Set?
- Mathematical Set Concept
- Why Sets Don't Allow Duplicates
- Hash-Based vs Tree-Based Sets
- Ordering Rules
- Equality & `hashCode()`
- Set Implementations (`HashSet`, `LinkedHashSet`, `TreeSet`)
- Internal Behavior
- Performance Analysis
- 20+ Interview Questions